THE PROPER HAT-GUESSING NUMBER OF THE FIVE-CYCLE: A COMPUTER-ASSISTED ARGUMENT

SHIQI LIU

Abstract. We present a computer-assisted argument that the proper hat-guessing number of the five-cycle is four. The main task is to exclude five arbitrary local guessing functions that win on every proper five-coloring. Counting strategy-table entries selects a color guessed between 17 and 25 times: the upper bound follows by averaging, and the lower bound follows from a classification of two-vertex covers of a missed-pair graph. Vertex and color relabeling then reduce the problem to 1413 count representatives, with selected representatives refined into 51 diagonal answer patterns. A complete local audit with a newly implemented checker replays all 8613 terminal RUP refutations, including 66,403,934 additions and every final empty clause. A new input auditor checks all 1413 parent formulas, all 7344 refined inputs, and all 65,399 valid counter inputs. We explain both the coverage of all candidate strategies and the extension of each candidate to its actual Boolean input; these are separate requirements from checking a refutation. An explicit local witness explains why the two-occurrence argument alone cannot give the full result. A concordance links the proof to the organized research and reproducibility records. This reimplementation audit remains part of the same AI-assisted workflow; external mathematical review and formal verification of the checking software remain outstanding.

1. Introduction and the main result

In the proper hat-guessing game, the players occupy the vertices of a graph, and the adversary assigns colors subject to the condition that adjacent vertices receive different colors. Each player observes the colors of its neighbors and simultaneously guesses its own color. A strategy is winning if at least one guess is correct for every permitted assignment. Adriaensen et al. introduced this variant and its parameter $\mathrm{HG}_P(G)$ in [1]. Their Table 1, in version 4, records $4 \le \mathrm{HG}_P(C_5) \le 5$. The task considered here is precisely to decide between these two values. The four-color construction is short; the five-color exclusion is the part that requires the finite computation.

This manuscript develops a computer-assisted argument for the following assertion.

Theorem 1.1 (Main theorem; computer-assisted)**.** *The proper hat-guessing number of the five-cycle is*

$$\mathrm{HG}_P(C_5) = 4.$$

The nonexistence argument concerns five arbitrary local functions. It does not assume that the players use the same rule, that left and right are interchangeable, or that a strategy is invariant under permutations of the vertices or colors. Symmetries are used only to select representatives of equivalent candidates.

The principal statistic is the number of local observations on which a fixed color is guessed. There are 125 entries in the five strategy tables. After discarding guesses that can never be correct, at least one color is guessed on at most 25 entries. Configurations in which this color occurs twice force its total count to be at least 17 and impose restrictions on the five individual counts. The resulting finite classification has 1413 representatives. Some representatives are refined using equality patterns of five diagonal answers, producing 8613

Date: October 26, 2026 (revision incorporating the complete local audit).

Key words and phrases. Proper hat guessing, cycle, finite classification, satisfiability, proof certificates.

Developed with computer and AI assistance. A complete local audit with reimplemented checkers has passed; external review and proof-assistant verification remain outstanding.

terminal Boolean formulas. The original local computation supplies a refutation of every terminal formula. The subsequent complete audit replays each refutation with a new checker and separately rechecks that the actual inputs and branching rules preserve every candidate winning strategy.

Why the argument has two parts. Fix a color and first consider only proper colorings in which it appears twice. This restricted family makes the interaction between two strategy tables visible and yields the lower bound 17. It does not characterize winning strategies for the whole game: a saved strategy attains 17 on this family but loses on colorings in the other two families. We include that witness in Appendix A, so the change from a local combinatorial argument to a full Boolean model has an explicit reason.

The proof has three obligations, each with a different kind of evidence:

(i) *Coverage:* every hypothetical winning strategy can be transformed into a listed branch. Sections 3 and 4 give the mathematical reductions.

(ii) *Input preservation:* that strategy would satisfy the actual branch formula after suitable auxiliary variables are assigned. Section 5 specifies the encoding and this extension check.

(iii) *Exclusion:* each terminal formula has a valid refutation. Section 6 gives the inference rule and identifies the recorded finite verification on which the conclusion depends.

A refutation alone establishes only the last obligation. Conversely, a complete list of branches does not establish that any branch is impossible.

The organization follows this logical dependence rather than the chronological order of the experiments. Section 2 fixes the model and normalization. Section 3 proves the combinatorial count restrictions. Section 4 establishes the complete finite classification. Sections 5 and 6 explain the Boolean encoding and certificate checks. Section 7 gives the final deduction. Appendix B connects the numbered results to the organized research archive and gives a procedure for independent reproduction.

Status of this manuscript. This manuscript was prepared with computer and AI assistance. On October 26, 2026, a new input auditor and a new RUP checker completed a full local audit of the adopted evidence. The combinatorial arguments were also examined, with exhaustive finite cross-checks; no error affecting the main conclusion was found in that audit. Its precise scope is given in Section 6.4 and Appendix C.

Here "independent implementation" means separate code, not independent human authorship or external peer review. The new auditor imports no earlier generator or auditor, and the new checker was written before the earlier checker sources were inspected. All implementations were nevertheless developed within the same AI-assisted workflow. The mathematical argument, source-to-model correspondence, and checking software remain open to external scrutiny; no proof-assistant verification is claimed. Bibliographic statements about [1] refer to the inspected version, not to an exhaustive assessment of subsequent literature.

2. The model and elementary reductions

We first specify the full strategy space. The later reductions will select equivalent candidates from this space, without assuming that the five players share a rule. Let $Q_q = \{1, \dots, q\}$. The vertices of C_5 are $1, \dots, 5$ in cyclic order; vertex indices are read modulo 5. Set

$$\mathcal{C}_q = \{c \in Q_q^5 : c_i \neq c_{i+1} \text{ for every } i\}.$$

In particular, $c_5 \neq c_1$ is required. Nonadjacent vertices may have the same color, and a member of $\mathcal{C}_q$ need not use all q colors. A strategy is a tuple $f = (f_1, \dots, f_5)$, where

$$f_i : Q_q^2 \longrightarrow Q_q.$$

The first and second arguments are the left and right neighbor colors, respectively. Winning means

$$\forall c \in \mathcal{C}_q \quad \exists i \in \{1, \dots, 5\} \quad f_i(c_{i-1}, c_{i+1}) = c_i. \tag{2.1}$$

There is no communication after the colors have been assigned. We use

$$\mathrm{HG}_P(C_5) = \sup\{q \in \mathbb{Z}_{\geq 2} : \text{a winning strategy with } q \text{ colors exists}\}.$$

The lower and upper bounds below make this supremum a finite maximum.

Lemma 2.1 (Restriction of the palette)**.** *If a winning strategy exists with q colors and $2 \leq r \leq q$, then a winning strategy exists with r colors.*

Proof. Choose an r-element subset R of the palette and restrict attention to configurations with colors in R. On observations in R^2, replace any answer outside R by an arbitrary element of R. Such an original answer could not have been correct on a configuration under consideration, so no previously correct guess is lost. Relabel R as Q_r. □

Proposition 2.2 (The four-color lower bound)**.** *There is a winning strategy on C_5 with four colors.*

Proof. Use the path strategy of [1, Lemma 4.1]. Identify the colors with $\mathbb{F}_2^2$ and write the colors at three consecutive vertices as a, b, c. Their guesses are, respectively,

$$(b_1 + 1, b_2), \qquad (c_1 + 1, a_2 + 1), \qquad (b_1, b_2 + 1).$$

All coordinates and additions in this construction are in $\mathbb{F}_2$. The end vertices ignore their other neighbor. Suppose both end guesses fail. Properness excludes $a = (b_1, b_2)$, and failure of the first guess excludes $a = (b_1 + 1, b_2)$. These are exactly the two colors with second coordinate b_2, so $a_2 = b_2 + 1$. Similarly, properness and failure of the third guess exclude (b_1, b_2) and $(b_1, b_2 + 1)$ for c; hence $c_1 = b_1 + 1$. Substituting these two identities into the middle guess gives $(b_1 + 1 + 1, b_2 + 1 + 1) = b$. The other two players may guess arbitrarily. □

Thus it suffices to exclude five colors. Henceforth $Q = Q_5$.

Notation	Meaning
$\mathcal{C}_q$	All proper colorings of the labeled cycle with palette Q_q.
M_i^z	The 4×4 indicator matrix for guessing z when neither neighbor has color z; row = left neighbor, column = right neighbor.
$s_i(z), S(z)$	The weight of this matrix and the sum of its weights over all five players.
a_j	The same five weights in the order $(1, 3, 5, 2, 4)$, in which nonadjacent players are consecutive.

The notation is defined formally below; this table fixes the two distinct orders used in the proof and in the saved data.

Lemma 2.3 (Removal of impossible guesses)**.** *A winning strategy can be replaced by one satisfying*

$$f_i(a, b) \notin \{a, b\} \qquad (i \in \{1, \dots, 5\},\ a, b \in Q). \tag{2.2}$$

Moreover, for a fixed color z, the replacement can preserve every indicator $\mathbf{1}[f_i(a, b) = z]$ with $a, b \neq z$.

Proof. If an answer belongs to $\{a, b\}$, it is incorrect in every proper coloring with that observation. Replacing it by any element of $Q \setminus \{a, b\}$ loses no correct guess. To preserve the indicated entries, put $S = Q \setminus \{z\}$. On an observation in S^2, replace an impossible answer by a color in $S \setminus \{a, b\}$; this set has at least two elements. Neither the old nor the new answer is z. On observations containing z, the specified indicators are not defined, so any allowed replacement may be used. Apply this independently to all affected entries. □

For $z \in Q$, define

$$M_i^z(a,b) = \mathbf{1}[f_i(a,b) = z] \quad (a,b \in Q \setminus \{z\}), \qquad s_i(z) = \sum_{a,b \neq z} M_i^z(a,b), \qquad S(z) = \sum_{i=1}^{5} s_i(z). \tag{2.3}$$

These counts refer to all five tables, not to the number of correct guesses on a particular coloring. Each $s_i(z)$ is the weight of a 4×4 Boolean matrix and lies between 0 and 16.

Lemma 2.4 (The total-count identity)**.** *For every strategy, $\sum_z S(z) \leq 125$. Under (2.2), equality holds. Consequently, a normalized winning strategy has a color z with $S(z) \leq 25$.*

Proof. There are five tables, each with 25 ordered observations. An entry $f_i(a,b) = g$ contributes to exactly one of the counts in (2.3) when $g \notin \{a,b\}$, namely to $s_i(g)$, and contributes to none otherwise. The final assertion is the pigeonhole principle. □

The fixed-color preservation in Lemma 2.3 has a specific quantifier: for each chosen z, there is a suitable repair. It does not assert that one repair preserves all five values $S(z)$ simultaneously. The main argument first makes any normalization and then chooses a least-used color. The preservation clause is needed when transferring a lower bound for a specified color back to an unnormalized strategy, as in Corollary 7.2.

The number of permitted configurations will also be useful. With J_q the all-ones matrix and I_q the identity matrix, proper cyclic colorings are the closed walks of length 5 in the color-transition matrix $J_q - I_q$. Its eigenvalues are $q-1$ once and -1 with multiplicity $q-1$, so

$$|\mathcal{C}_q| = \operatorname{tr}((J_q - I_q)^5) = (q-1)^5 - (q-1). \tag{2.4}$$

In particular, $|\mathcal{C}_5| = 1020$ and $|\mathcal{C}_4| = 240$. Vertices are labeled, so there is no division by 5. A fixed color can occur zero, one, or two times on C_5; the corresponding numbers are

$$240, \qquad 5 \cdot 4 \cdot 3^3 = 540, \qquad 5 \cdot 4 \cdot (4 \cdot 3) = 240. \tag{2.5}$$

Indeed, one occurrence leaves a path on four vertices, and two occurrences must occupy one of the five nonadjacent pairs.

3. A combinatorial lower bound for a fixed color

Fix $z \in Q$ and put $S = Q \setminus \{z\}$. By Lemma 2.3, we may work with a normalized strategy without changing the matrices M_i^z. Throughout this section it is enough to require winning only on the configurations in which z occurs twice. The argument proceeds from one fixed nonadjacent pair of z-vertices to all five such pairs. We first obtain a pairwise inequality, then analyze equality, and finally show that total weights 15 and 16 would force incompatible equality cases.

3.1. **A missed-pair graph.**

Fix two nonadjacent vertices $i, i+2$ and write a configuration in cyclic order starting at i as

$$(z,u,z,v,w), \qquad u,v,w \in S, \quad v \neq w. \tag{3.1}$$

Set

$$t = f_{i+1}(z,z), \qquad \alpha(w) = f_{i+3}(z,w), \qquad \beta(v) = f_{i+4}(v,z).$$

Normalization gives $t \in S$, $\alpha(w) \in S \setminus \{w\}$, and $\beta(v) \in S \setminus \{v\}$. Define

$$E = \{(v,w) \in S^2 : v \neq w,\ \alpha(w) \neq v,\ \beta(v) \neq w\}. \tag{3.2}$$

The variables u, v, w describe actual hat colors, whereas t, α, β are answers supplied by the strategy. The five observations give:

Player	Hat	Observation	Condition for a correct guess
i	z	(w,u)	$M_i^z(w,u)=1$
$i+1$	u	(z,z)	$u=t$
$i+2$	z	(u,v)	$M_{i+2}^z(u,v)=1$
$i+3$	v	(z,w)	$\alpha(w)=v$
$i+4$	w	(v,z)	$\beta(v)=w$

View E as a bipartite graph whose row vertices are labeled by v and whose column vertices are labeled by w. Among the 12 ordered unequal pairs, α guesses exactly four and β guesses exactly four. Their sets may intersect, so

$$|E| \geq 4. \tag{3.3}$$

Every vertex of this graph has degree at most 3; in particular, one vertex cannot cover all its edges. Notice that E does not depend on u. Even when a row and a column have the same color label, they are distinct vertices of this bipartite graph. A vertex cover is a set of row and column vertices meeting every edge; it is not a set of colors.

For $u \neq t$, the middle player in (3.1) is wrong. For $(v,w) \in E$, both end players are wrong as well. Thus one of the two players wearing z must be correct, which is equivalent to

$$M_i^z(w,u)=1 \quad \text{or} \quad M_{i+2}^z(u,v)=1. \tag{3.4}$$

For a fixed u, the selected columns w and selected rows v must therefore form a vertex cover of E. For this fixed u, its size is exactly the number of selected entries in column u of M_i^z plus the number in row u of M_{i+2}^z. The selected entries for different values of u are disjoint. Thus the three cover bounds may be added without double counting.

Lemma 3.1 (Pair inequality)**.** *For every i,*

$$s_i(z) + s_{i+2}(z) \geq 6.$$

Consequently, $S(z) \geq 15$.

Proof. For each of the three choices $u \in S \setminus \{t\}$, a vertex cover as in (3.4) has at least two vertices by (3.3). Hence

$$\sum_{w \in S} M_i^z(w,u) + \sum_{v \in S} M_{i+2}^z(u,v) \geq 2.$$

Sum over these three values of u and include the nonnegative contribution at $u=t$. This gives the pair inequality. Summing it over the five values of i counts every $s_i(z)$ twice and gives $2S(z) \geq 30$. □

3.2. **All vertex covers of size two.** The equality case of Lemma 3.1 requires more information than the number of edges in E.

Lemma 3.2 (Two-vertex covers)**.** *For a graph E defined by* (3.2)*, the following hold.*

(i) *A cover consisting of a row a and a column b is possible only when $a=b$. If it exists, it is the unique cover of size two, and E is the disjoint union of two copies of $K_{1,2}$ centered at that row and column.*

(ii) *There is at most one cover consisting of two rows and at most one consisting of two columns.*

(iii) *If both pure covers exist, E consists of all four edges of the resulting 2×2 rectangle, and its row-label and column-label sets are disjoint.*

Proof. Suppose that row a and column b, with $a \neq b$, cover E. The submatrix on $S \setminus \{a\}$ and $S \setminus \{b\}$ contains nine pairs. Exactly two are diagonal pairs, since $|S \setminus \{a,b\}| = 2$, leaving seven permitted unequal pairs. None may belong to E. However, α can cover at most one pair in each of the remaining three columns, and β can cover at most one in each of the three rows. Together they cover at most six, a contradiction.

Now suppose that row a and column a cover E, and let $T = S \setminus \{a\}$. The six unequal pairs in T^2 must all be covered by α or β. Each function covers at most three such pairs. Thus both cover exactly three, their covered sets are disjoint, and $\alpha(T), \beta(T) \subseteq T$. In column a, the three legal rows lie in T. The function β guesses a at none of these rows, while $\alpha(a) \in T$ covers exactly one of the three pairs. Exactly two edges of E remain in column a. In row a, the function α never answers a in columns of T, and $\beta(a) \in T$ covers exactly one of its three legal cells. Exactly two edges therefore remain in row a as well. There are no other edges. The two stars have disjoint vertex sets. A cover of a two-edge star that uses only one vertex must use its center, so their two centers are the unique cover of size two.

If two distinct pairs of rows each cover E, every row incident with an edge belongs to the intersection of the two pairs, which has size at most one. This would give at most three edges, contradicting (3.3). The same reasoning applies to columns. Finally, if a pair of rows and a pair of columns both cover E, all edges lie in their 2×2 rectangle. There are at least four edges, so every cell is an edge. A shared color label between the two pairs would put a forbidden diagonal cell in this rectangle. The label sets must therefore be disjoint. □

3.3. **Tight pairs and consecutive equalities.** The support of a Boolean matrix is called an $r \times s$ full rectangle if it is the product of an r-element row set and an s-element column set. In the following statement a rectangle with a zero dimension is empty.

Lemma 3.3 (Structure of a tight pair)**.** *If $s_i(z) + s_{i+2}(z) = 6$, then either*

(i) *the supports of M_i^z, M_{i+2}^z are full rectangles of sizes 1×3 and 3×1, respectively; or*

(ii) *for some $k \in \{0, 1, 2, 3\}$ they are full rectangles of sizes $2 \times k$ and $(3-k) \times 2$, respectively.*

In particular,

$$(s_i(z), s_{i+2}(z)) \in \{(0, 6), (2, 4), (3, 3), (4, 2), (6, 0)\}. \tag{3.5}$$

Proof. In the proof of Lemma 3.1, the three slices $u \neq t$ must each contribute exactly two entries, and the slice $u = t$ must contribute zero. Each selected pair is therefore a cover of E of size two. There are no additional nonzero entries outside these covers: their three contributions already account for the entire weight six.

If a mixed cover exists, Lemma 3.2 makes it unique: row a and column a for some $a \in S$. Thus $M_i^z(a, u) = 1$ for all $u \neq t$, and these are its only nonzero entries. Likewise, the only nonzero entries of M_{i+2}^z are $M_{i+2}^z(u, a) = 1$ for $u \neq t$. This gives (i).

If there is no mixed cover, each slice chooses either the unique pure column cover, when available, or the unique pure row cover. Let U be the set of slices using the column cover and put $k = |U|$. The nonzero entries of M_i^z have row labels equal to the two selected column labels of E and column labels in U. The nonzero entries of M_{i+2}^z have row labels in $(S \setminus \{t\}) \setminus U$ and column labels equal to the two selected row labels of E. This proves (ii), including the cases in which only one pure cover is available. When $k = 0$ or $k = 3$, the unused cover need not exist; the corresponding matrix is empty, and its zero-dimensional rectangle denotes just the empty support. Counting entries gives (3.5). □

Corollary 3.4 (A restriction at two tight pairs)**.** *If $s_{i-2}(z) + s_i(z) = s_i(z) + s_{i+2}(z) = 6$, then $s_i(z) \in \{0, 4\}$.*

Proof. By (3.5), the candidates are $0, 2, 3, 4, 6$. In the pair $(i-2, i)$ the matrix M_i^z is the right-hand matrix of Lemma 3.3, whereas in the pair $(i, i+2)$ it is the left-hand matrix. For weights 2, 3, and 6, its required dimensions in these two roles are, respectively,

weight	2	3	6
right-hand role	1×2	3×1	3×2
left-hand role	2×1	1×3	2×3

A fixed nonempty matrix cannot have two different numbers of nonzero rows or columns. These three weights are excluded, leaving only 0 and 4. □

Proposition 3.5 (The lower bound 17)**.** *Every five-color winning strategy satisfies* $S(z) \geq 17$ *for each* $z \in Q$*.*

Proof. Put $a_j = s_{1+2j}(z)$ for $j \in \mathbb{Z}/5\mathbb{Z}$. Thus the player order is $(1, 3, 5, 2, 4)$, and $a_j + a_{j+1} \geq 6$. Lemma 3.1 already gives $\sum_j a_j \geq 15$.

If $\sum_j a_j = 15$, all five pair inequalities are equalities. The equations $a_{j+1} = 6 - a_j$ around an odd cycle force every $a_j = 3$, contradicting Corollary 3.4.

If $\sum_j a_j = 16$, put $e_j = a_j + a_{j+1} - 6$. Summing gives $\sum_j e_j = 2\sum_j a_j - 30 = 2$. Such a nonnegative integer vector has either one entry 2 or two entries 1. In the latter case their positions on a five-cycle are adjacent or nonadjacent; rotations and reflections identify all choices of each kind. These are exactly three cases. Alternating the five pair equations gives

$$2a_0 = (6 + e_0) - (6 + e_1) + (6 + e_2) - (6 + e_3) + (6 + e_4), \quad a_{j+1} = 6 + e_j - a_j.$$

The following table keeps the same index choices in both columns:

$(e_0, e_1, e_2, e_3, e_4)$	$(a_0, a_1, a_2, a_3, a_4)$	Forbidden center
$(2, 0, 0, 0, 0)$	$(4, 4, 2, 4, 2)$	$a_2 = 2$
$(1, 1, 0, 0, 0)$	$(3, 4, 3, 3, 3)$	$a_3 = 3$
$(1, 0, 1, 0, 0)$	$(4, 3, 3, 4, 2)$	$a_4 = 2$

Each indicated center has both incident pair sums equal to six, but its value is neither 0 nor 4. Corollary 3.4 excludes all three cases. Thus 16 is impossible as well. The count-preserving part of Lemma 2.3 extends the conclusion to strategies that were not initially normalized. □

3.4. **What the local bound does and does not prove.** The argument above uses only 240 of the 1020 proper colorings. The value 17 is sharp for that restricted requirement: Appendix A gives explicit tables winning on all two-occurrence colorings and having $S(5) = 17$. That same completion loses on other colorings. Thus a larger lower bound for a full winning strategy must use additional information, including compatibility with the zero- and one-occurrence families. The next sections do this by keeping every proper coloring in the base formula while fixing only the five counts. They do not freeze the positions or the non-special answers of the local witness.

4. Complete finite classification

We now retain only the necessary information proved in the preceding section. The resulting classes are deliberately larger than the set of winning strategies. Excluding all of them will therefore exclude every winning strategy, even though some listed vectors may be unrealizable for reasons not used in the classification.

4.1. **Count representatives.** For an integer n, let $\mathcal{A}_n$ be the set of integer vectors $a = (a_0, \ldots, a_4)$ satisfying

$$0 \leq a_j \leq 16, \qquad \sum_{j=0}^{4} a_j = n, \tag{4.1}$$

$$a_j + a_{j+1} \geq 6, \tag{4.2}$$

$$a_j + a_{j+1} = 6 \implies (a_j, a_{j+1}) \in \{(0,6), (2,4), (3,3), (4,2), (6,0)\}, \tag{4.3}$$

$$a_{j-1} + a_j = a_j + a_{j+1} = 6 \implies a_j \in \{0, 4\}. \tag{4.4}$$

All indices in these conditions are modulo 5. These conditions are necessary, not asserted to be sufficient for a strategy.

Let D_5 act on these vectors by cyclic shifts and reversal. Choose the representative that minimizes the ordered pair

$$(a_0 + a_1, (a_0, a_1, a_2, a_3, a_4)) \tag{4.5}$$

lexicographically, and denote the set of representatives by $\mathcal{R}_n$. The first component of (4.5) is an ordering convention that places a least pair sum first; it is not an additional restriction. After one representative has been selected from each orbit, the saved list is sorted lexicographically by the five-tuple itself. This serialization order is distinct from the key used to select a representative; set equality and absence of duplicates determine coverage.

Lemma 4.1 (Validity of the graph reduction)**.** *Every winning strategy with $S(z) = n$ has an equivalent strategy whose count vector in the order $(1, 3, 5, 2, 4)$ belongs to $\mathcal{R}_n$.*

Proof. The count vector belongs to $\mathcal{A}_n$ by the results of Section 3. A rotation of the vertices transports the five functions in their cyclic order. A reflection transports the functions and interchanges their left and right arguments. In either case proper colorings are carried bijectively to proper colorings, and correct guesses are preserved. On the matrices, reflection includes transposition, which preserves their weights. The action on the order $(1, 3, 5, 2, 4)$ is again all cyclic shifts and reversals: multiplication by 2 is invertible modulo 5. Thus an automorphism sends the count vector to its representative without requiring any invariance of the strategy itself. □

For clarity, use vertex labels in $\mathbb{Z}/5\mathbb{Z}$ temporarily. A rotation $h(i) = i + d$ transports rules by $g_{h(i)}(u, v) = f_i(u, v)$; a reflection $h(i) = d - i$ transports them by $g_{h(i)}(u, v) = f_i(v, u)$. For $c'_{h(i)} = c_i$, both formulas give $g_{h(i)}(c'_{h(i)-1}, c'_{h(i)+1}) = f_i(c_{i-1}, c_{i+1})$. Thus each individual correct guess is preserved. In the count order $i = 1 + 2j$, the induced actions are $j \mapsto j + k$ and $j \mapsto k - j$.

There are two elementary ways to produce the entire list. One enumerates $(a_0, a_1, a_2, a_3) \in \{0, \dots, 16\}^4$, sets $a_4 = n - a_0 - a_1 - a_2 - a_3$, and checks (4.1)–(4.4). A different enumeration uses slack variables

$$e_j = a_j + a_{j+1} - 6, \qquad e_j \geq 0, \qquad \sum_j e_j = 2n - 30. \tag{4.6}$$

Write $b_j = 6 + e_j$. The equations $a_j + a_{j+1} = b_j$ have the unique solution

$$2a_0 = b_0 - b_1 + b_2 - b_3 + b_4, \qquad a_{j+1} = b_j - a_j \quad (0 \leq j < 4). \tag{4.7}$$

Enumerating all compositions of $2n - 30$ into five nonnegative parts, applying (4.7), and testing (4.1)–(4.4) therefore gives exactly the same set $\mathcal{A}_n$. An odd numerator in (4.7) is rejected. The last cyclic equation follows from the formula for a_0; the total sum is also checked explicitly in the audit. This establishes completeness of both enumeration schemes.

For example, the list at weight 17 consists of

$$\mathcal{R}_{17} = \{(2, 4, 3, 3, 5),\ (2, 4, 4, 2, 5),\ (3, 3, 4, 3, 4)\}.$$

The middle vector specifies player counts $(2, 2, 4, 5, 4)$ in the ordinary order $(1, 2, 3, 4, 5)$, but fixes none of the locations of the counted entries. It is also the count vector of the local witness in Appendix A.

4.2. **Color relabeling and 51 diagonal patterns.** A permutation σ of Q acts on strategies by

$$f_i^\sigma(a, b) = \sigma\big(f_i(\sigma^{-1}(a), \sigma^{-1}(b))\big). \tag{4.8}$$

It preserves properness and correctness of each guess. After making the chosen color equal to 5, permutations of the remaining four colors preserve every $s_i(5)$. In particular, since $f_1(5, 5) \neq 5$, we may impose

$$f_1(5, 5) = 1. \tag{4.9}$$

No second normalization involving a different observation is imposed in the count-branch inputs.

Lemma 4.2 (Diagonal-pattern reduction)**.** *Let $t_i = f_i(5,5) \in \{1,2,3,4\}$. Up to a common permutation of these four colors, $(t_1,\dots,t_5)$ is one of the 51 words satisfying*

$$t_1 = 1, \qquad 1 \le t_j \le \min\{4, 1 + \max_{\ell<j} t_\ell\} \quad (2 \le j \le 5). \tag{4.10}$$

This normalization preserves the count representative already chosen.

Proof. Assign the label 1 to the first color encountered in the word, label 2 to the next previously unseen color, and so forth. Extend this assignment arbitrarily to a permutation of all four colors. The resulting word satisfies (4.10). Conversely, its equality classes determine it uniquely. These words correspond to partitions of five labeled positions into at most four nonempty blocks. The recurrence $\left\{{n \atop k}\right\} = k\left\{{n-1 \atop k}\right\} + \left\{{n-1 \atop k-1}\right\}$ gives

$$\left\{{5 \atop 1}\right\} + \left\{{5 \atop 2}\right\} + \left\{{5 \atop 3}\right\} + \left\{{5 \atop 4}\right\} = 1 + 15 + 25 + 10 = 51.$$

The permutation fixes color 5, so it only permutes the 16 observations inside each M_i^5 and preserves its weight. If (4.9) already holds, the first-occurrence relabeling fixes its answer 1. □

Proposition 4.3 (Coverage of all five-color candidates)**.** *If a five-color winning strategy exists, there are $n \in \{17,\dots,25\}$ and $a \in \mathcal{R}_n$ such that an equivalent normalized winning strategy has count vector a and satisfies* (4.9)*. For any such count branch selected for refinement, at least one of the 51 constraints* (4.10) *still contains an equivalent winning strategy.*

Proof. Normalize by Lemma 2.3, choose a color with total at most 25 by Lemma 2.4, and relabel it as 5. Proposition 3.5 supplies the lower bound 17. Apply Lemma 4.1, then a color permutation fixing 5 to obtain (4.9). If the branch is refined, apply Lemma 4.2. Each step preserves winning and all count conditions needed by the next step. □

The 51 subcases need not exhaust all labeled diagonal words as literal assignments. They cover them up to the explicitly proved color action. This distinction is essential: the proof does not impose a symmetry condition on the unknown functions.

The two reductions are applied in a fixed order. First a vertex automorphism selects the count representative; next a permutation of the four non-special colors selects a diagonal representative. The second operation leaves every count unchanged, so it cannot undo the first. There is no requirement that the diagonal tuple also be minimal under vertex automorphisms.

5. Boolean encoding and preservation of candidates

A formula in conjunctive normal form (CNF) is an AND of clauses, each clause being an OR of literals. Our base variables describe actual answers; auxiliary variables are used only to encode numerical counts. A genuine strategy supplies the base values, and we must show that suitable auxiliary values exist.

5.1. **The base formula.** For $i,a,b,g \in Q$, with i denoting a vertex and $g \notin \{a,b\}$, introduce a Boolean variable

$$x_{i,a,b,g} \quad \text{meaning} \quad f_i(a,b) = g.$$

At each observation (i,a,b) impose the clauses

$$\bigvee_{g \notin \{a,b\}} x_{i,a,b,g}, \tag{5.1}$$

$$\neg x_{i,a,b,g} \vee \neg x_{i,a,b,h} \quad (g < h,\ g,h \notin \{a,b\}). \tag{5.2}$$

For every $c \in \mathcal{C}_5$, add the coverage clause

$$\bigvee_{i=1}^{5} x_{i,c_{i-1},c_{i+1},c_i}. \tag{5.3}$$

All its variables exist because c is proper. Let Φ be the conjunction of these clauses.

Lemma 5.1 (Exactness of the base encoding)**.** *The satisfying assignments of* Φ *are in bijection with normalized five-color winning strategies.*

Proof. Clauses (5.1) and (5.2) select exactly one allowed answer at each observation, determining a unique normalized strategy. Clause (5.3) holds precisely when at least one player guesses correctly on its indexed configuration. There is one such clause for every member of $\mathcal{C}_5$, so their conjunction is exactly (2.1). Conversely, a normalized winning strategy sets its selected answer variables to true and satisfies all these clauses. □

For each player there are five equal-color observations with four allowed answers and twenty unequal-color observations with three allowed answers. The formula therefore has

$$5(5 \cdot 4 + 20 \cdot 3) = 400$$

variables. The number of selection clauses is

$$5\left[5\left(1+\binom{4}{2}\right)+20\left(1+\binom{3}{2}\right)\right] = 575.$$

Together with the 1020 coverage clauses, this gives 1595 clauses. In particular, equal-color neighbor observations and colorings using fewer than five colors have not been omitted.

5.2. **The count-branch formula.** Fix $a \in \mathcal{R}_n$, and write $r_i = a_j$ when $i = 1 + 2j$ modulo 5. The intended additional constraints are

$$\sum_{u,v\in\{1,2,3,4\}} x_{i,u,v,5} = r_i \quad (1 \le i \le 5), \qquad x_{1,5,5,1} = 1. \tag{5.4}$$

They fix only five counts and one permissible normalization. In particular, no positions of the r_i selected entries and no other answer values are fixed.

The recorded implementation uses PySAT sequential-counter components for the five equalities [2]. Each component has exactly the 16 listed input variables and its own auxiliary variables. Denote the actual generated component by $K_{i,r_i}(X_i, Y_i)$, where $X_i = (x_{i,u,v,5})_{u,v\in\{1,2,3,4\}}$ and the Y_i are disjoint. The actual parent formula is

$$F_a = \Phi \wedge x_{1,5,5,1} \wedge \bigwedge_{i=1}^{5} K_{i,r_i}(X_i, Y_i). \tag{5.5}$$

For a refined parent, the formula at pattern $t \in \mathcal{T}$, where $\mathcal{T}$ is defined by (4.10), is exactly

$$F_{a,t} = F_a \wedge \bigwedge_{i=1}^{5} x_{i,5,5,t_i}. \tag{5.6}$$

The first of these five units duplicates the normalization in F_a and is harmless.

For the weight-17 example $a = (2, 4, 4, 2, 5)$, the counter targets are $(r_1, r_2, r_3, r_4, r_5) = (2, 2, 4, 5, 4)$. In a possible refinement such as $t = (1, 2, 1, 3, 2)$, the answers at $(5, 5)$ are fixed to those five values, while every other answer remains subject only to the base formula and counts. Fixing a count is therefore different from fixing a table.

5.3. **Why auxiliary encodings do not discard a true strategy.** The required direction for an impossibility proof is

$$\Big(\sum X_i = r_i\Big) \implies \exists Y_i \; K_{i,r_i}(X_i, Y_i). \tag{5.7}$$

Even if a component were to admit an unintended input, its unsatisfiability in the full branch would still exclude all intended inputs. Thus the reverse implication is not needed for this argument. Rather than relying only on a library's claim that an encoding is exact, the recorded audit checks (5.7) on every intended input of each actual component type.

This one-way implication is essential. Accidentally removing an intended input could invalidate an exclusion proof. Admitting extra inputs makes exclusion harder, but an actual

refutation would still exclude the intended ones. We do not assert a bijection between strategies and assignments of auxiliary variables; Lemma 5.1 concerns only the base formula.

For this check, the 16 input variables are renamed $1, \dots, 16$, and the component's auxiliary variables are renamed consecutively from 17. A component type is identified by its count and its complete normalized clause list, not just by the count. After a particular input assignment has been substituted, each remaining clause has at most one positive auxiliary literal. This Horn property is checked from the actual clauses.

Under substitution, a clause with a true input literal is discarded; false input literals are removed from the remaining clauses. A remaining clause $\neg y_1 \vee \cdots \vee \neg y_k \vee y$ is the rule $y_1 \wedge \cdots \wedge y_k \Rightarrow y$. With no positive literal its head is $\bot$; with no negative literals it is a fact. This also covers unit clauses and empty residual clauses.

Lemma 5.2 (Auxiliary extension check)**.** *For a fixed input assignment, suppose the residual auxiliary formula is Horn. Start with no auxiliary variable true. Repeatedly apply every rule*

$$y_{j_1} \wedge \cdots \wedge y_{j_k} \implies y_\ell$$

whose premises are true. Treat a clause with no positive literal as the same rule with conclusion $\bot$. If closure is reached without deriving $\bot$, setting exactly the derived variables true satisfies the residual formula.

Proof. There are finitely many variables, and every successful application adds a previously false variable, so the process terminates. At closure, a rule with all premises true has its head true. A rule with head $\bot$ cannot have all premises true. Hence every residual Horn clause is satisfied. Clauses already satisfied by the fixed input remain satisfied, so this assignment also satisfies the original component. □

The implementation additionally evaluates every original component clause on the constructed assignment. For count r, it enumerates every r-element subset of the 16 input positions, with no sampling. It checks the variable ranges and clause intervals to ensure that no component uses another component's auxiliary variables or unrelated strategy variables. Consequently, the five separate auxiliary assignments can be combined.

Proposition 5.3 (Preservation by a branch input)**.** *Suppose the base clauses, variable lists, extra units, and all properties* (5.7) *have been checked for a branch. Every normalized winning strategy in that branch extends to a satisfying assignment of its actual formula* (5.5) *or* (5.6)*.*

Proof. Lemma 5.1 supplies a satisfying assignment of Φ. The strategy satisfies the branch counts and the normalization, so (5.7) supplies an auxiliary assignment for each component. The auxiliary sets are disjoint, so their union satisfies all five components simultaneously. In a refined branch the strategy also satisfies its five diagonal units. □

The input audit reconstructs (5.1)–(5.3) directly from the game, compares the base clause multiset, checks all five component boundaries, and confirms that the only other parent clause is $x_{1,5,5,1}$. Every child is compared clause by clause with its parent plus the five specified units. These checks address the mathematical input to the solver, separately from any subsequent refutation.

6. Refutations and the recorded finite verification

The reductions now specify a finite list of formulas. A solver searches for refutations, but its reported status alone is not the evidence used here. Each adopted proof is a sequence of clauses checked by a separate program using the elementary rule below.

6.1. **The inference rule checked.**

A literal is a Boolean variable or its negation, and a clause is a disjunction of literals. Given a set of clauses D, unit propagation repeatedly sets the sole unassigned literal of a clause to true when all its other literals are false. An empty clause or conflicting unit assignments produce a contradiction. For example, from $(p \vee q)$ and

$\neg p$, assuming $\neg q$ forces p and then a conflict. Hence the unit clause q has the RUP property defined below. For an empty added clause there are no assumptions: the retained database must itself produce a unit-propagation conflict.

Definition 6.1. A clause $C = \ell_1 \vee \cdots \vee \ell_k$ has the *reverse-unit-propagation* (RUP) property with respect to D if unit propagation on

$$D \wedge \neg\ell_1 \wedge \cdots \wedge \neg\ell_k$$

produces a contradiction. The same definition applies when C is empty.

Lemma 6.2 (Soundness of RUP checking)**.** *If C is RUP with respect to D, then $D \models C$. Starting from a formula F, any sequence of RUP additions and deletions that reaches a checked empty clause proves that F is unsatisfiable, provided all clauses used in each check are original clauses or previously verified consequences of F.*

Proof. Every assignment satisfying a unit clause must make its unique remaining literal true. Induction over propagation steps shows that a satisfying assignment of $D \wedge \neg C$ would have to obey all propagated units. It cannot obey a contradictory pair or satisfy an empty clause. Hence $D \wedge \neg C$ is unsatisfiable and $D \models C$.

Inductively, every retained clause is a consequence of F. A newly accepted clause follows from some retained consequences of F, so it too is a consequence of F. Deleting clauses preserves this invariant. A checked empty clause therefore implies $F \models \bot$. □

This is a standard clausal proof mechanism; see [3] for the wider DRUP/DRAT setting. The checks used here accept only RUP additions, not general RAT additions. They are custom local implementations, not an execution of the external DRAT-trim program.

In the original verification, the fast checker permanently retains original input clauses and processes deletion records for learned clauses. When it rejects a certificate, a second checker starts from the original input, ignores all deletions, and checks every addition anew. Either accepted execution is sound by Lemma 6.2: retaining previously established consequences cannot introduce an unjustified premise. A rejected addition is never skipped. Parser checks require valid variable ranges and clause terminators for the input and all additions, the declared input clause count, and a checked empty clause. The deletion-ignoring checker skips deletion records; it does not validate their literal lists. This does not affect its inference argument: a deletion record neither adds a premise nor removes an obligation to check an addition.

6.2. **The finite assertion and its local evidence.**

Proposition 6.3 (Finite exclusion by computation)**.** *For every $17 \le n \le 25$ and $a \in \mathcal{R}_n$, either the parent F_a has a RUP refutation, or all 51 formulas $F_{a,t}$, $t \in \mathcal{T}$, have RUP refutations. The associated input-extension and coverage checks described above pass for the recorded inputs.*

Recorded computational verification. The two enumeration schemes following (4.7) produce identical representative lists. The Boolean inputs are checked as in Proposition 5.3. Refutations are generated by Glucose3 through PySAT, then read from disk and checked using the separate RUP implementations. The later complete audit replays all terminal refutations with `RupAudit`, checking every addition and the final empty clause afresh. It also rechecks input preservation using a new standard-library auditor. Table 1 gives the complete branch counts and accepted addition counts. The full certificate files, input lists, source fingerprints, and audit reports are retained in the local research archive described in Appendix B. These original and subsequent checks supply the computational component of the argument. The external-review and software-verification limitations stated in the introduction continue to apply. □

TABLE 1. The complete recorded classification and final certificate set. $N_n = |\mathcal{A}_n|$, $R_n = |\mathcal{R}_n|$, B_n is the number of refined parents, and L_n is the number of terminal certificates.

n	N_n	R_n	B_n	L_n	RUP additions
17	20	3	0	3	14,413
18	90	12	0	12	70,751
19	250	29	0	29	284,448
20	526	60	0	60	742,441
21	940	102	0	102	2,776,244
22	1,515	163	11	713	5,704,885
23	2,275	240	28	1,640	11,590,970
24	3,245	341	36	2,141	16,112,383
25	4,451	463	69	3,913	29,107,399
Total	13,312	1,413	144	8,613	66,403,934

Every refined parent is replaced by all 51 subcases, so

$$L_n = (R_n - B_n) + 51B_n = R_n + 50B_n. \tag{6.1}$$

Thus there are 1269 direct terminal certificates and $144 \cdot 51 = 7344$ refined terminal certificates. Superseded direct certificates and duplicate precomputations are not counted again.

The audit of the diagonal reduction separately enumerates all 1024 raw words and checks the action of all 24 permutations of the old colors on the base clauses and the five count-input sets. The local mathematical argument for this reduction remains Lemma 4.2; this enumeration is an implementation check.

For the last two weights, the count-extension checks cover counts $0, \dots, 12$ and $0, \dots, 13$, respectively. The corresponding numbers of valid input assignments are

$$\sum_{r=0}^{12} \binom{16}{r} = 64,839, \qquad \sum_{r=0}^{13} \binom{16}{r} = 65,399.$$

The audit records for the earlier weights give the remaining figures in Table 2. These are exhaustive checks of the actual component types occurring in each weight, not random tests of a few assignments.

TABLE 2. Reported valid-input extension checks, by total weight. Assignments for component types used at different weights may be checked more than once.

n	17	18	19	20	21	22	23	24	25
Checks	6,868	14,877	26,333	39,203	50,643	58,651	63,019	64,839	65,399

6.3. **Limits, failures, and the scope of the evidence.** The initial searches at weights 24 and 25 used a budget of 100,000 conflicts per parent and left 36 and 69 parents undecided. Each was then refined into all 51 patterns, with a budget of 300,000 conflicts per subcase. The final refutations cover every such subcase. Other weights also used refinement for selected difficult or large-certificate parents. A budget exhaustion, an abnormal solver exit, or an unverified `UNSAT` status is not used as an exclusion.

In the original audit, the accepted deletion-ignoring fallback was used for 129 terminal certificates at weights 22–23 and 296 at weights 24–25. Those counts refer to full replays, not omitted proof lines. Separate comparisons with simple propagation implementations are retained in the archive, but finite implementation tests are not a proof-assistant verification of the checker.

The complete evidence manifest records 30,110 files totaling 6,617,692,938 bytes, with SHA-256 hashes. These figures concern the selected evidence files, not every historical output in the workspace. Hash equality establishes file integrity, not mathematical correctness. Some final audit passes reused already checked terminal certificates only when the input, proof, and checker hashes matched. The records distinguish such reuse from a fresh replay. In particular, no reported final-pass runtime should be interpreted as the time for an independent reproduction from scratch.

6.4. **Complete audit with reimplemented checkers.** The later audit completed on October 26, 2026, at 21:38:40 in UTC+08:00. It read the adopted CNF and proof files directly, without using old successful-check caches. All 8613 terminal proofs passed, with 66,403,934 verified additions and a checked empty clause for each input. The per-leaf identifiers were matched to the complete terminal list with no missing or duplicate leaf, and the addition counts agree with Table 1 at every weight.

The new Python auditor checked all 1413 parent inputs and all 7344 refined child inputs against the game and branch definitions. It checked 14 actual component types on all 65,399 valid input assignments and evaluated every component clause on each constructed auxiliary assignment. It also matched hashes for the 17,226 terminal CNF and proof files, totaling 6,585,642,510 bytes. This file set excludes the extra metadata counted in the older 30,110-file manifest.

TABLE 3. Complete new replay. A full fallback restarts the proof from its original input and checks all additions while ignoring deletions.

n	Parents	Leaves	RUP additions	Full fallbacks
17	3	3	14,413	0
18	12	12	70,751	0
19	29	29	284,448	2
20	60	60	742,441	6
21	102	102	2,776,244	16
22	163	713	5,704,885	47
23	240	1,640	11,590,970	82
24	341	2,141	16,112,383	111
25	463	3,913	29,107,399	185
Total	1,413	8,613	66,403,934	449

The new checker first retains every original clause and processes deletions of learned clauses. If an addition fails this check, it restarts from the original input and ignores all deletions; it never skips a rejected addition. Its parser validates input and proof literal ranges and terminators, including deletion records, checks the declared input clause count, and requires the final checked empty clause. The inference invariant is precisely Lemma 6.2.

There were 449 full fallbacks: 24 at weights 19–21, 129 at weights 22–23, and 296 at weights 24–25. In the original verification, weights 17–21 used the deletion-ignoring checker directly, so no corresponding preliminary fallback count was reported. The new total therefore does not contradict the earlier figures.

The audit also checked 6561 endpoint-function pairs, 1440 tight-support cases, the full count classification, all 1024 diagonal words, all 24 permutations of the non-special colors, and all 10 graph automorphisms. It directly verified the four-color formula on all 240 proper colorings and the displayed local witness on all 1020 five-colorings. The new checker passed 150 seeded comparisons with a simple propagation implementation and 20 deterministic boundary or resolution tests. These are finite implementation checks, not a formal correctness proof of the checker. No new SAT search was needed for this audit.

7. The deduction and the status of the argument

We first isolate the precise logical role of the computational assertion.

Theorem 7.1 (Certificate-to-game implication)**.** *If the finite exclusion and input checks in Proposition 6.3 are correct, then there is no five-color winning strategy on C_5.*

Proof. Suppose a winning strategy exists. Proposition 4.3 places an equivalent normalized winning strategy in a parent count branch with $17 \le n \le 25$. If the parent is a direct terminal branch, Proposition 5.3 extends this strategy to a satisfying assignment of F_a, whereas Proposition 6.3 and Lemma 6.2 make F_a unsatisfiable.

If the parent is refined, Lemma 4.2 first places an equivalent strategy in one of its 51 subcases. This transformation preserves the parent counts and its normalization. The same extension argument gives a satisfying assignment of that $F_{a,t}$, contradicting its checked refutation. These two cases exhaust the parent list. □

Argument for Theorem 1.1. The original verification and the complete new replay supply the premise of Theorem 7.1, and hence excludes five colors, subject to the verification status stated in this manuscript. Proposition 2.2 provides a winning strategy for four colors. If a winning strategy existed for more than five colors, Lemma 2.1 would produce one for five colors. Therefore the value established by this computer-assisted argument is exactly four. □

There is also an informative intermediate consequence of excluding the weights below 25.

Corollary 7.2 (The last remaining count)**.** *Assume that the exclusions at weights 17–24 are correct. Any five-color winning strategy would then satisfy*

$$S(1) = S(2) = S(3) = S(4) = S(5) = 25$$

and would have no entry that guesses a neighbor's color. The verified exclusion at weight 25, if correct, contradicts this necessary condition.

Proof. For any fixed z, the count-preserving normalization of Lemma 2.3, followed by the same graph and color reductions, would place a strategy with $S(z) \le 24$ in an excluded branch; Proposition 3.5 rules out totals below 17. Thus every $S(z) \ge 25$. Lemma 2.4 gives $\sum_z S(z) \le 125$ even before normalization. Equality must hold in all five lower bounds, and every entry must contribute to the sum. An entry guessing a neighbor's color would contribute zero, so no such entry can occur. Weight 25 is then excluded by the final family of certificates. □

This equality concerns the total for a color over all five players; it does not say that each player guesses each color five times. It is a necessary condition inside the proposed contradiction, not a construction of a winning strategy. Moreover, none of the branch inputs used another color's lower bound or the conclusion of another weight. The finite exclusions therefore do not depend circularly on this balance statement.

Computer and AI assistance. OpenAI Codex assisted with mathematical exploration, program development, local computations, organization of the evidence, and preparation and revision of this manuscript. The solver, input auditors, and proof checkers have different roles, but were integrated within the same AI-assisted workflow. The status distinction made in the introduction therefore applies throughout: separate local programs are not external reviewers, and neither the written argument nor the checking software has been formally verified in a proof assistant.

Data availability and further work. The organized research archive contains definitions, proof drafts, programs, inputs, certificates, audit reports, and historical failures. A private repository copy and versioned evidence attachments exist for the earlier manuscript and research archive. The complete audit package and the present revised manuscript have been organized locally; their creation is not a claim of public availability or external review. The appendix gives paths relative to the research root, so readers with access can trace each assertion to its source and reproduce the checks. Manuscript source alone is insufficient for a full replay: the large certificate files must also be present.

A full local replay with newly implemented software has now been completed. The next verification tasks are external scrutiny of the combinatorial reductions and use of a separately reviewed or formally verified certificate checker. A further mathematical goal is to replace the large finite exclusion with a shorter argument explaining the incompatibility between the three occurrence families. The local witness shows why the 240 two-occurrence colorings alone cannot supply that explanation.

Appendix A. A local optimum that is not a full winning strategy

This example comes from the saved two-occurrence experiment of October 24, 2026 (research Lemma L010). It explains why the local bound 17 and the subsequent exclusions for the full game concern different requirements. It is not used as a premise of any branch refutation.

Put $z = 5$ and $S = \{1, 2, 3, 4\}$. For each player let

$$P_i = \{(a, b) \in S^2 : f_i(a, b) = 5\}, \quad A_i(w) = f_i(5, w), \quad B_i(v) = f_i(v, 5), \quad t_i = f_i(5, 5).$$

The four entries of each A_i and B_i are listed in the order $1, 2, 3, 4$. The following data specify every entry involving color 5 and every old observation whose answer is 5:

i	P_i	A_i	B_i	t_i
1	$\{(1,2),(2,2)\}$	$(3,3,1,1)$	$(2,4,1,1)$	1
2	$\{(3,1),(3,3)\}$	$(3,3,4,1)$	$(2,4,4,2)$	4
3	$\{(1,3),(1,4),(3,3),(3,4)\}$	$(2,4,4,2)$	$(3,4,4,1)$	4
4	$\{(1,3),(2,1),(2,2),(2,3),(3,3)\}$	$(2,1,1,2)$	$(3,3,1,1)$	1
5	$\{(2,2),(2,4),(4,2),(4,4)\}$	$(2,4,4,1)$	$(4,3,4,3)$	4

For $(a, b) \in S^2 \setminus P_i$, complete the table by $f_i(a, b) = \min(S \setminus \{a, b\})$. This set is nonempty, and all 125 entries are now specified. Direct inspection of the displayed data shows that every answer differs from its observed neighbor colors. The special-color counts are $(2, 2, 4, 5, 4)$, whose sum is 17.

Here is a finite check stated without dependence on a SAT solver. For each nonadjacent pair $i, i + 2$, examine all $u, v, w \in S$ with $v \neq w$. There are $4 \cdot 4 \cdot 3 = 48$ triples for each of five pairs. In each triple, at least one of

$$u = t_{i+1}, \quad A_{i+3}(w) = v, \quad B_{i+4}(v) = w, \quad (w, u) \in P_i, \quad (u, v) \in P_{i+2}$$

holds. The revision check enumerates these 240 triples directly and also compares all 125 reconstructed answers with the saved JSON tables. Together with the hand lower bound in Proposition 3.5, this finite witness check establishes the local optimum 17 for the two-occurrence requirement.

The same direct evaluation over all 1020 proper colorings gives:

Occurrences of color 5	0	1	2
Proper colorings	240	540	240
All five guesses wrong	35	55	0

For example, $(2, 3, 2, 3, 4)$ is proper and produces the five guesses $(1, 1, 5, 1, 1)$, so every player is wrong. This refutes the claim that this particular completion is winning on the full game. It does not exclude every completion of its sets P_i, or every strategy with total 17. Those are

stronger assertions requiring their own evidence. The full-game count branches avoid this distinction by retaining all 1020 coverage clauses and allowing every table position to vary.

The original files are `strategy.json` and `independent-verification.json` in the directory

`attempts/A001-v4-extension/checks/outputs/double-layer-bound17/`

The earlier revision's check is preserved in `20261026-rigorous-revision/checks/mathematical-review.json` under `paper/versions/`. The full audit's new witness check is recorded in `raw/math-report.json` and `raw/supplement-report.json` in the package described in Appendix C.

Appendix B. Evidence organization and reproduction

This appendix documents the original computation and the earlier partial revision checks. The later complete local audit is documented in Appendix C; no external replay is claimed.

B.1. **From the paper to the organized research record.** The paper follows the dependence of the final argument. The archive also preserves the order in which methods were tried; these two orders should not be confused. The research identifiers L001–L020 denote entries in `lemmas.md`; R008 denotes `results/R008-five-color-impossibility.md`. They are stable archive identifiers, not theorem numbers in this paper.

Paper result	Research source	What to examine
Model and reductions, Section 2	L001, L002, L007, L008, L012	Original quantifiers, ordered observations, palette restriction, and count-preserving normalization.
Pair and tight-pair lemmas, Section 3	L009–L010; A001 proof draft	Missed-pair covers, rectangle supports, and the distinction between a local optimum and a full-game strategy.
Count and diagonal coverage, Section 4	L015–L016; A005 proof draft	The player-order conversion, canonical representatives, and a single common color permutation.
Input preservation, Section 5	`models/sat.md`; A001 input audits	All proper colorings, actual counter inputs, disjoint auxiliaries, and absence of unintended restrictions.
Finite exclusion and deduction, Sections 6–7	L018–L020; R008	The adopted leaf set, RUP steps, and the exact conditional passage from a refuted formula to the game.
Local witness, Appendix A	L010; A001 two-occurrence experiment	The complete table and the differing outcomes in the three families.

The two attempt directories are

A001: `attempts/A001-v4-extension/`
A005: `attempts/A005-diagonal-pattern-exhaustion/`

Their `proof.md` and `log.md` files preserve derivations and failures. The four principal review entry points are:

- `review/proof-map.md`: argument dependencies and evidence roles;
- `review/catalog/README.md`: the nine weights, their parents, and the finally adopted terminal certificates;
- `review/reproduction.md`: commands that leave original evidence intact;
- `review/environment.md`: versions, file formats, and index conventions.

Two historical methods clarify what the final argument does not need. The earlier position-pattern proof at weights 17–19 fixes all 80 special-color indicators. The final count proof redoes these weights with only five counts and does not depend on that earlier core chain. The A005 experiment, which used the 51 diagonal classes without count constraints, returned

UNKNOWN in every class under its recorded budget. The final proof uses those classes only inside selected count branches. Neither the earlier restricted failure nor those unknown statuses are substitutes for a terminal refutation.

B.2. **A traceable branch and its refinement.** A branch identifier is local to its weight, and starts at zero. At weight 17, `profile-001/` corresponds to $a = (2, 4, 4, 2, 5)$ in order $(1, 3, 5, 2, 4)$, hence to $(2, 2, 4, 5, 4)$ in the player order used by `groups.json`. The local witness in Appendix A has these counts but violates some base coverage clauses, so it is not a satisfying assignment of this branch. Its local success and this branch's refutation are therefore consistent.

At weight 24, `profile-317/` is a refined parent. To inspect it, first locate the parent in `review/catalog/branches.json`, check its input and group metadata, and then follow all 51 entries under `diagonal-split/`. For instance, `case-11111/` fixes all five answers at $(5, 5)$ to 1. The saved parent summary records 51 leaves and 336,715 accepted additions in total. An old parent UNKNOWN report is not the final status of these child refutations.

B.3. **Recorded environment and file layout.** The recorded runs used Windows, Python 3.13.0, the Python package `python-sat` version 1.9.dev17, and the Glucose3 interface `g3`. The RUP checkers were written in C# and compiled with the Microsoft .NET Framework compiler. The profile enumeration, diagonal enumeration, and input-extension checks are exhaustive and deterministic; no random seed is used in these enumerations. Historical randomized implementation cross-checks have their own recorded seeds and are not used in place of the exhaustive checks.

Paths below are relative to the research root: locally it is `P001-c5-five-color-hats`, and in the prepared repository copy it is `research/`. The numbering and relative evidence paths agree. The main computational directory is

`attempts/A001-v4-extension/checks/`.

For each weight n, its subdirectory `outputs/weightNN-count-profiles/` contains

(i) `profiles.json`, the ordered list of count representatives;

(ii) a directory `profile-NNN/` for each representative, with `instance.cnf`, `groups.json`, and the original solver report;

(iii) either a direct `proof.drup`, or a `diagonal-split/` directory listing all 51 patterns and their child inputs and certificates;

(iv) terminal `certificate-audit.json` records and parent `profile-certificate-audit.json` records;

(v) `independent-audit.json`, which records the complete input, coverage, extension, and refutation audit for the weight.

The generator, count-input auditor, and proof checker are distinct programs. The principal source files are listed by role below.

Role	Source files
Input generation	`solve_count_profiles.py`
	`split_diagonal_profiles.py`
Input and coverage audits	`audit_count_profiles_native.py`
	`audit_count_profiles_adaptive.py`
	`profile_certificate_audit.py`
RUP checking	`RupCheckNative.cs`
	`RupCheckActive.cs`

The three successive stage manifests are

Weights	Manifest
17–21	`count-profile-stage.json`
22–23	`adaptive-count-stage.json`
24–25	`final-count-stage.json`

Their combined file manifest is `complete-proof-stage.json`. Its recorded SHA-256 fingerprint is

```
1d41cffdaf48dbba3be5a0b9f064503bdb64
946b5bf6062b1052af22b2f4da6e
```

where the displayed lines are concatenated without a separator. This fingerprint identifies one local manifest; it is not a proof that its contents are mathematically sound. The manifest includes fingerprints of the actual checker executables and sources as well as the input and proof files.

The separate four-color data are `checks/outputs/q4-strategy.json` and `checks/outputs/q4-final-verification-20261025.json`. A direct configuration verifier, which does not import the SAT encoder, reports 240 legal configurations and no failure.

B.4. **An explicit enumeration procedure.** The following finite procedure is sufficient to reconstruct every count representative without trusting the original list.

(i) For each $n = 17, 18, \ldots, 25$, enumerate all $(e_0, \ldots, e_4) \in \mathbb{Z}^5_{\geq 0}$ with total $2n - 30$.

(ii) Compute $b_j = 6 + e_j$ and the candidate a by (4.7). Reject a nonintegral candidate or one failing (4.1)–(4.4).

(iii) Form the five shifts of a and the five shifts of its reversal. Retain their minimum under (4.5); remove duplicate minima.

(iv) Sort the representatives by their five-tuples (ordinary lexicographic order), compare with `profiles.json`, and check the corresponding row of Table 1.

Formula (4.7) proves that no candidate satisfying the count conditions is missed. For the refinement, enumerate all words in $\{1, 2, 3, 4\}^5$, replace each word by its first-occurrence labels, remove duplicates, and compare the resulting 51 words with the stored list.

B.5. **What an independent replay must establish.** An independent replay should verify the following logically separate requirements.

(i) Reconstruct the 125 one-hot answer choices and all 1020 proper configurations, and match the exact base formula of every parent input.

(ii) Check the semantic identity of each of the five groups of 16 count variables, the intended count, the auxiliary-variable boundaries, and the absence of unlisted constraints. Establish (5.7) for every actual component type by exhaustive input substitution and direct clause evaluation, or by a separately justified encoder proof.

(iii) Reconstruct both symmetry actions, the representative list, and every selected 51-way refinement. Confirm that a child has exactly the parent clauses and its specified diagonal units.

(iv) Read every adopted proof, verify every addition by unit propagation, and require a checked empty clause. Ignoring all deletion records is permitted by Lemma 6.2; accepting an unchecked addition is not.

(v) Check that all 1413 parents are covered by the 8613 terminal certificates, with no unknown or unverified terminal case. Verify the four-color lower bound separately, either by the displayed construction or by direct evaluation of all 240 configurations.

The recorded audit scripts implement these checks, but a rerun that merely accepts stored cache reports is not a fresh verification of the proof steps. For a genuinely fresh replay, each adopted input and proof must be passed directly to a checker. In the recorded implementation a terminal call has the form

```
RupCheckNative.exe instance.cnf proof.drup
```

and the alternative fast checker has the same arguments. A zero exit code is accepted only together with the checked-empty-clause report. A new implementation or a separately verified proof checker would give a stronger independence guarantee than replaying the same executable.

B.6. **Bookkeeping boundaries.** Raw solver statuses, interrupted runs, and later replacements are kept separate. In particular, some parent reports remain `UNKNOWN` because their exclusion was subsequently obtained from all their refined children. Some control-program completion reports were not updated after a standard-output encoding error, although the independent mathematical audit had already written its completed report. These events are recorded separately; they are not treated as refutations. The final selected-input and certificate manifest, together with the complete per-weight audit, identifies the evidence actually used in the proposed proof.

Source and document snapshots do not duplicate all large certificate files. Reproduction therefore requires the original input and certificate directories as well as the manuscript sources and manifest. No claim is made that the present PDF, on its own, constitutes the complete machine certificate archive.

B.7. **Historical checks for the earlier 19-page revision.** The earlier October 26, 2026 revision added checks whose outputs are separate from the historical computation. The new standard-library script `checks/check_mathematics.py` in the preserved `20261026-rigorous-revision/` manuscript directory does not import the old generator or auditor. It enumerates all $3^8 = 6561$ pairs (α, β), checks the two-cover classification and all 1440 resulting tight-support cases, reconstructs the 1413 count representatives, and checks all 1024 diagonal words. It also evaluates the displayed four-color formula on all 240 proper colorings and reconstructs the local witness on all 1020 five-colorings. These are finite cross-checks of the text, not substitutes for its general deductions.

During that earlier revision, the existing read-only reproduction tool was run on all 1413 parent inputs and all 7344 refined child inputs. Its input audit checked 14 component types and 65,399 valid input assignments; it uses pure functions from the earlier auditor and is not a new independent implementation. Fresh proof replay covered the three weight-17 leaves and all 51 children of weight-24 parent 317, totaling 54 certificates and 351,128 additions. That was a limited replay, not a repeat of all 66,403,934 historical additions.

The corresponding outputs are in the following directories under `review/runs/`:

```
20261026-paper-revision-inputs/
20261026-paper-revision-direct-replay/
20261026-paper-revision-split-replay/
```

Each `report.json` records its exact command, environment, scope, status, and elapsed time. The revision also checks manuscript tables against the saved per-weight audit reports. Compilation and page inspection are reported separately from all mathematical checks.

Appendix C. The complete audit record

The full reimplementation audit is separate from the earlier revision's 54-certificate replay. Its raw outputs remain in

`checks/independent-review-20261026/`.

A frozen, organized copy is provided at

`review/audits/20261026-full-replay/`.

Paths here are relative to the research root. The organized package contains 372 original audit files copied byte for byte, the adopted parent and terminal indexes, a joined per-leaf

audit ledger, and new reproduction instructions. Packaging these records is distinct from running the audit again.

C.1. **Records and their roles.** The package root contains a Chinese audit report and manuscript revision notes. Its principal machine-readable files are listed below.

File	Role
`data/summary.json`	Scope and aggregate counts of the completed audit.
`data/leaf-audit.jsonl`	One record for each of the 8613 adopted leaves, joining the input/proof paths and hashes to its fresh replay result.
`data/weight-summary.json`	Counts, checked additions, and full fallbacks at each weight.
`data/branches.json`	All 1413 parents, count vectors, and refinement choices.
`raw/replay-0.tsv`–`raw/replay-3.tsv`	Original per-leaf outputs from the four checking processes.
`raw/inputs-report.json`	All parent/child input and auxiliary-extension checks.
`raw/source-change-audit.json`	Recorded comparison of the two input-auditor source snapshots.
`package-manifest.json`	Sizes and SHA-256 hashes of the packaged files; the manifest itself is excluded from its own list.

The four checking processes finished successfully. Each output row was written only after that proof reached its checked empty clause. The summary then verified equality with the adopted terminal list and the expected per-leaf addition counts. Durations recorded per leaf include any complete fallback; their sum is not the wall time of the parallel run.

C.2. **Source provenance and implementation checks.** The new checker is `RupAudit.cs`; its SHA-256 is

`8ddc76a5a8a154bc32ee16668f50d5259e7`
`bc9cb669cce6c68205ffd618f8344.`

The displayed lines are concatenated without the final punctuation. The full audit uses this checker, not the experimental `RupAuditRoot.cs` variant, which is retained only as a separate trial. The original audit environment was Windows 11, Python 3.13.0, and the Microsoft .NET Framework C# compiler. No SAT library was imported by the new auditor or checker. The main enumerations and replays are deterministic; seed 20261026 applies only to the 150 comparison tests.

One source-provenance detail is recorded explicitly. The input-audit process started with the source now saved as `review-initial.py`. While it ran, only the count-list comparison in a different function, `local_math`, was corrected: the initial test had confused the representative-selection key with the list's serialization order. The input-audit report read its source hash from disk at process exit, so that field identifies the corrected file rather than the file loaded at process start. Both snapshots are retained. A subsequent syntax-tree comparison confirms that `inputs` and all its helpers are identical between them. The discrepancy is a provenance issue in the report, not an omitted input check; it should not be concealed by rewriting the old report's hash.

C.3. **Reproduction with new output directories.** The organized package's `tools/verify_package.py` checks its own file inventory and hashes and the recorded audit totals. It does not recheck mathematical inferences. The separate `tools/replay.py` accepts an explicit research-root directory and a fresh output directory, recompiles the preserved checker, checks selected input/proof hashes, and passes the actual files to it. It records new compiler output, commands, fingerprints, per-leaf results, and a scope-limited report. Its `-full` option selects all leaves; weight, profile, and limit options permit smaller checks whose reports remain explicitly limited. The original run files and batch paths are preserved as historical data.

The compact audit package contains code, tests, reports, hashes, and evidence indexes. The 6,585,642,510 bytes of terminal CNF and proof files remain in the research archive at the indexed relative paths. They are required for proof replay and are not embedded in the compact package or the manuscript source archive. Hash matching alone establishes integrity, not the correctness of the mathematical argument.

References

[1] S. Adriaensen, P. Bentley, A. Bishnoi, W. Cames van Batenburg, M. Kreiger, L. van der Kuil, S. Mandal, A. Ramachandran, and J. Tuite, *Hat guessing with proper colorings*, arXiv:2603.04909v4 [math.CO], 2026. https://arxiv.org/abs/2603.04909v4.

[2] A. Ignatiev, A. Morgado, and J. Marques-Silva, *PySAT: A Python toolkit for prototyping with SAT oracles*, Theory and Applications of Satisfiability Testing–SAT 2018, Springer, 2018, pp. 428–437. https://doi.org/10.1007/978-3-319-94144-8_26.

[3] N. Wetzler, M. J. H. Heule, and W. A. Hunt, Jr., *DRAT-trim: Efficient checking and trimming using expressive clausal proofs*, Theory and Applications of Satisfiability Testing–SAT 2014, Lecture Notes in Computer Science **8561**, Springer, 2014, pp. 422–429. https://www.cs.utexas.edu/~marijn/drat-trim/.

Dalian University of Technology